

评分：_____

南京信息工程大学

《面向对象程序设计实践》课程
综合实训报告

题 目 Campus RainHub

学 院： 计算机学院

班 级： 23 计科 3 班

小组编号： 第 41 组

任课教师： 耿学华

学 期： 2025-2026 (1)

Campus RainHub 校园智能共享雨具系统

目 录

1 系统分析	2
1.1 应用系统简介.....	2
1.1.1 课题背景与研究意义.....	2
1.1.2 可行性研究与环境约束.....	2
1.2 系统需求（功能）分析.....	2
1.2.1 客户端（Client）功能需求.....	2
1.2.2 管理端（Admin）功能需求.....	3
2 系统设计	3
2.1 总体设计	3
2.1.1 MVC 架构模式	3
2.1.2 系统模块图.....	3
2.1.3 系统模块说明表.....	4
2.2 详细设计	5
2.2.1 数据结构设计.....	5
2.2.2 主要类设计.....	6
3 系统实现	9
3.1 系统开发环境.....	9
3.2 程序编码	9
3.2.1 后端工作概述.....	9
3.2.2 前端工作详述.....	9
4 系统测试	12
4.1 用户借伞流程测试.....	12
4.2 用户还伞流程测试.....	13
4.3 管理员操作流程测试.....	13
5 总结与体会	14
5.1 遇到的问题及解决方法.....	14
5.2 系统优点	14
1. 完整的前后端架构.....	14
2. 现代 C++特性应用	14
3. 工程化实践.....	14
5.3 系统不足	15
5.4 收获与体会	15
5.5 致谢	15
参考文献:	15

1 系统分析

1.1 应用系统简介

1.1.1 课题背景与研究意义

本系统旨在解决校园生活中常见的痛点问题：突发降雨时无伞可用的困境、传统借伞需人工值守服务时间受限、雨伞丢失损坏难以追溯以及多站点借还管理复杂等问题。通过引入智能终端和自助服务，提高校园生活的便利性。

1.1.2 可行性研究与环境约束

技术可行性分析：

技术维度	技术选型	可行性说明
开发语言	C++17	高性能、跨平台、面向对象特性完善
GUI 框架	Qt 6.x	成熟的跨平台 GUI 框架，信号槽机制便于事件处理
数据库	MySQL 8.0	关系型数据库，事务支持完善，适合业务数据管理
构建系统	CMake 3.16+	跨平台构建，支持模块化编译

表 1-1 技术可行性分析表

环境约束：

- 操作系统：Windows 10/11、Linux、macOS
- 编译器：支持 C++17 标准的编译器（MSVC 2019+、GCC 9+、Clang 10+）
- Qt 版本：Qt 6.2.0 及以上
- MySQL 版本：MySQL 8.0 及以上
- 内存需求：最低 512MB 可用内存

1.2 系统需求（功能）分析

本系统分为客户端（Client）和管理端（Admin）两个部分，分别面向普通用户和管理员。

1.2.1 客户端（Client）功能需求

功能需求：

功能模块	功能描述
用户认证	支持刷卡登录、学号登录、首次激活设置密码、密码修改
借伞功能	查看站点可借雨具、选择槽位借取、实时库存刷新
还伞功能	选择空槽位归还、自动计费、余额扣除
个人中心	查看个人信息、余额查询、刷新余额
站点地图	可视化展示校园站点分布、站点库存预览
使用说明	服务协议、收费标准、使用指南

表 1-2 客户端功能需求分析表

非功能需求：

- 界面响应时间 < 500ms

- 支持 3 秒自动刷新槽位状态
- 美观现代的深蓝渐变主题界面
- 友好的用户交互提示

1.2.2 管理端（Admin）功能需求

功能需求：

功能模块	功能描述
管理员认证	管理员账号密码登录、权限验证
首页概览	设备在线率、借出数量、故障数统计、站点雨具概览表格
雨具管理	按站点/槽位筛选雨具、修改雨具状态（可借/故障/已借出）
用户管理	用户搜索、用户信息查看、密码重置
订单流水	查看最近借还记录、费用统计

表 1-3 管理端功能需求分析表

非功能需求：

- 5 秒自动刷新当前页面数据
- 表格支持排序和筛选
- 操作结果即时反馈

2 系统设计

2.1 总体设计

2.1.1 MVC 架构模式

本系统采用经典的 MVC（Model-View-Controller）架构模式。View 层负责 Qt 界面的展示与交互，Controller 层（Service）处理业务逻辑，Model 层（DAO）负责数据对象封装与数据库访问，实现了界面与业务逻辑的分离，提高了代码的可维护性和可扩展性。



图 2-1 MVC 架构概念图

2.1.2 系统模块图

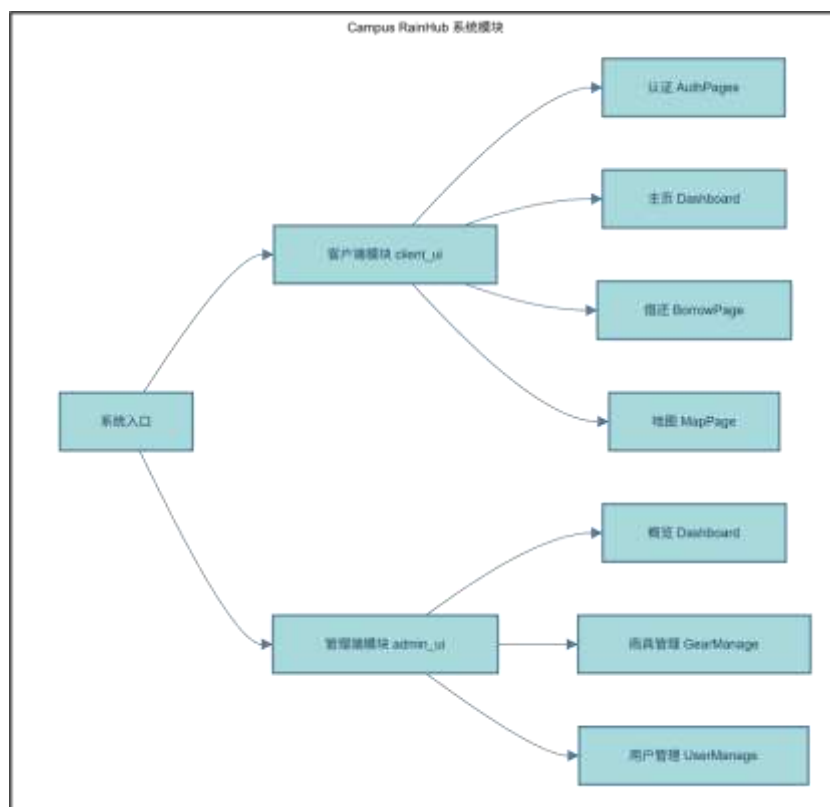


图 2-2 系统模块图

2.1.3 系统模块说明表

表现层模块如下表所示：

模块名称	所属层	核心功能
MainWindow (Client)	表现层	客户端主窗口，页面调度器，管理 11 个页面的切换与数据传递
AdminMainWindow	表现层	管理端主窗口，管理 5 个管理页面，侧边栏导航
WelcomePage	表现层	系统欢迎页，展示 Logo 和"开始使用"入口
AuthPages	表现层	认证页面集合，包含刷卡、登录、注册、改密等 5 个子页面
DashboardPage	表现层	客户端主页，站点选择、借/还伞入口、导航菜单
BorrowPage	表现层	借还伞页面，3×4 槽位网格、实时状态刷新、借还操作
MapPage	表现层	站点地图页，可视化展示校园 14 个站点分布
SlotItem	表现层	槽位组件，自定义 QWidget，展示雨具图标和状态
Styles	表现层	全局样式定义，深蓝渐变主题 QSS 样式表

表 2-1 表现层模块说明表

控制层模块如下表所示：

模块名称	所属层	核心功能
AuthService	控制层	用户认证服务，登录验证、注册激活、密码管理
BorrowService	控制层	借还服务，借伞/还伞业务逻辑、费用计算
StationService	控制层	站点服务，获取站点列表、站点详情、库存统计

表 2-2 控制层模块说明表

数据访问层模块如下表所示：

模块名称	所属层	核心功能
UserDao	数据访问层	用户数据访问，CRUD 操作封装
GearDao	数据访问层	雨具数据访问，按站点/状态查询
RecordDao	数据访问层	借还记录数据访问，订单管理
ConnectionPool	数据访问层	数据库连接池，线程本地连接管理

表 2-3 数据访问层模块说明表

模型层模块如下表所示：

模块名称	所属层	核心功能
User	模型层	用户实体类，封装用户属性和方法
RainGear	模型层	雨具抽象基类，定义雨具公共接口
GlobalEnum	模型层	全局枚举定义，GearType、GearStatus、Station

表 2-4 模型层模块说明表

2.2 详细设计

2.2.1 数据结构设计

1. 数据库表设计

数据库表设计与规范如下：

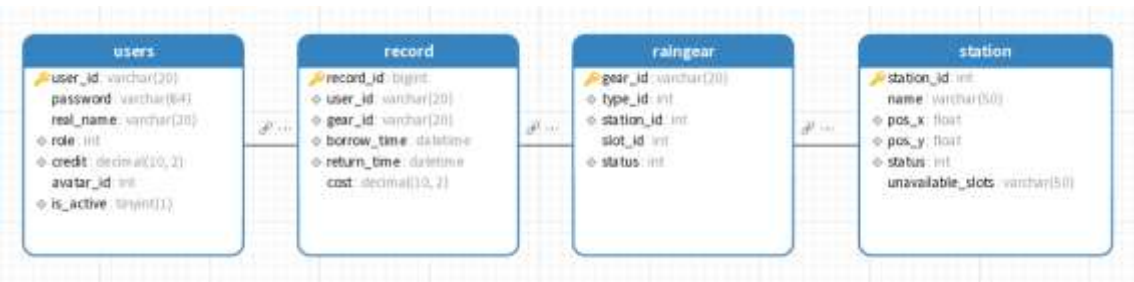


图 2-3 数据库表结构与约束图

2. 枚举类型设计

```

01 // 雨具类型枚举
02 enum class GearType {
03     Unknown = 0,
04     StandardPlastic = 1,    // 普通塑料伞
05     PremiumWindproof = 2,  // 高质量抗风伞
06     Sunshade = 3,          // 专用遮阳伞

```

```

07     Raincoat = 4           // 雨衣
08 };
09
10 // 雨具状态枚举
11 enum class GearStatus {
12     Unknown = 0,
13     Available = 1, // 可用
14     Borrowed = 2, // 借出
15     Broken = 3     // 损坏
16 };
17
18 // 站点枚举 (14 个校园站点)
19 enum class Station {
20     Unknown = 0,
21     Wende, Mingde, Library, Changwang, Oufang, Beichen,
22     Dorm1, Dorm2, Dorm3, Dorm4, Dorm5, Dorm6,
23     Gym, Admin
24 };

```

3. 服务返回结构结构体

```

1 // 借还操作结果
2 struct ServiceResult {
3     bool success; // 操作是否成功
4     QString message; // 提示信息
5     double cost = 0.0; // 产生的费用
6 };

```

4. 地图配置 JSON 结构

地图数据使用 JSON 配置文件存储，便于后期维护和扩展站点信息：

```

01 {
02     "stations": [
03         {
04             "station_id": 1,
05             "name": "文德楼",
06             "pos_x": 0.20,
07             "pos_y": 0.40,
08             "description": "主要教学楼"
09         }
10         // ... 共 14 个站点
11     ]
12 }

```

2.2.2 主要类设计

系统核心类如下：

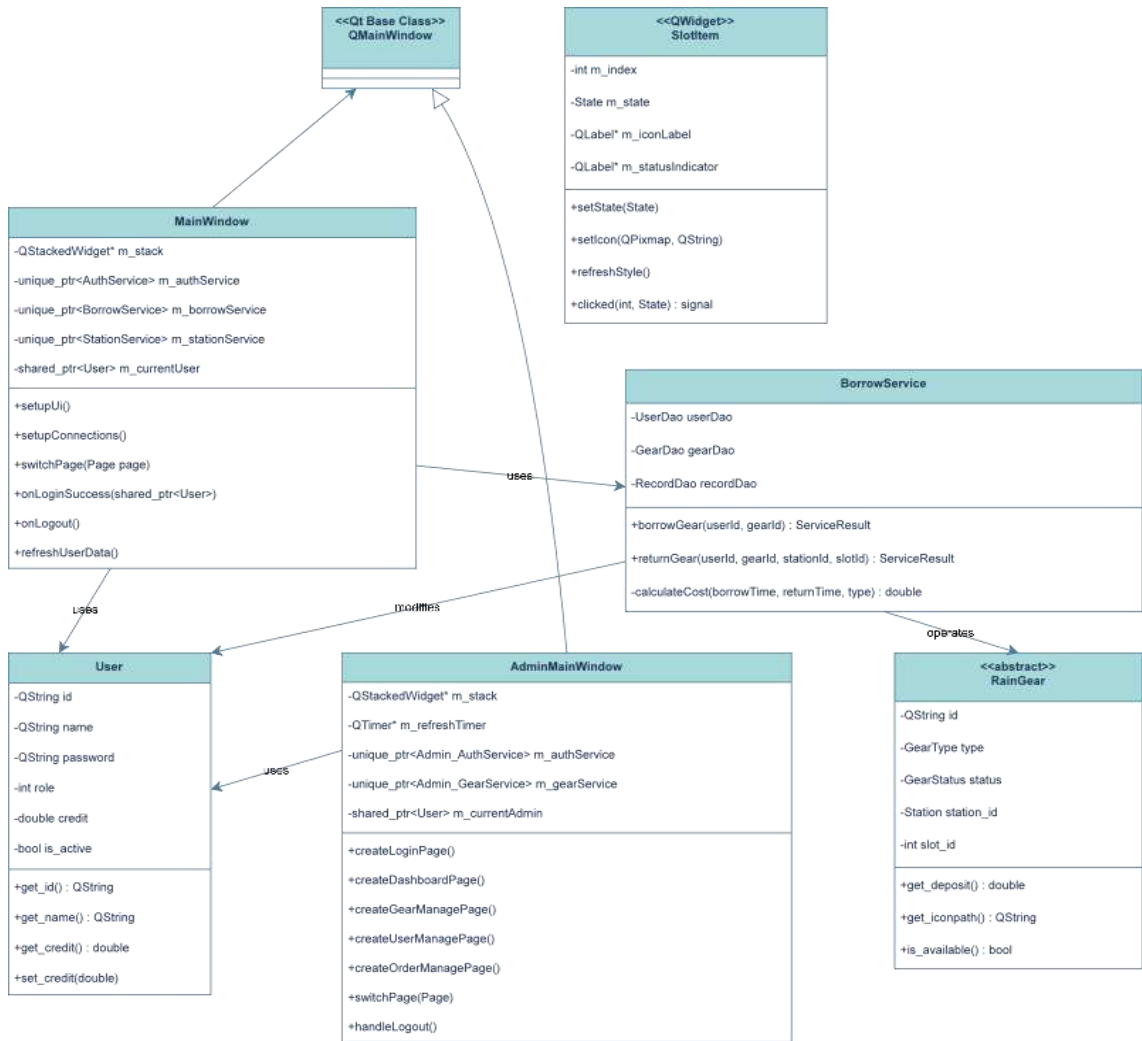


图 2-4 系统核心类图

抽象类设计说明：

类名	类型	职责说明	关键方法/属性
RainGear	抽象基类	雨具的共同基类，定义雨具的公共属性和纯虚接口	get_deposit(), get_iconpath() 为纯虚函数
StandardPlasticUmbrella	派生类	普通塑料伞，押金 5 元	实现 get_deposit() 返回 5.0
PremiumWindproofUmbrella	派生类	高质量抗风伞，押金 15 元	实现 get_deposit() 返回 15.0
SunshadeUmbrella	派生类	专用遮阳伞，押金 10 元	实现 get_deposit() 返回 10.0
Raincoat	派生类	雨衣，押金 8 元	实现 get_deposit() 返回 8.0

表 2-4 模型层模块说明表

前后端接口如下：

1. BorrowService 借还服务接口

方法签名	参数说明	返回值	功能描述
borrowGear(userId, gearId)	userId: 用户学号 gearId: 雨具 ID	ServiceResult	执行借伞操作, 更新雨具状态和借还记录
returnGear(userId, gearId, stationId, slotId)	userId: 用户学号 gearId: 雨具 ID stationId: 归还站点 slotId: 归还槽位	ServiceResult	执行还伞操作, 计算费用并扣款
calculateCost(borrowTime, returnTime, type)	borrowTime: 借出时间 returnTime: 归还时间 type: 雨具类型	double	根据使用时长和雨具类型计算费用

表 2-5 BorrowService 接口规范

2. StationService 站点服务接口

方法签名	参数说明	返回值	功能描述
getAllStations()	无	vector<shared_ptr<StationLocal>>	获取所有站点列表
getStationDetail(stationId)	stationId: 站点枚举	shared_ptr<StationLocal>	获取站点详细信息, 包含槽位状态
getStationStats()	无	vector<StationStats>	获取所有站点统计信息 (管理端)
getOnlineRate()	无	double	获取设备在线率百分比

表 2-6 StationService 接口规范

3. AuthService 认证服务接口

方法签名	参数说明	返回值	功能描述
verifyUser(userId, name)	userId: 学号 name: 姓名	optional<User>	验证用户身份, 返回用户信息
login(userId, password)	userId: 学号 password: 密码	optional<User>	用户登录验证
register(userId, password)	userId: 学号 password: 新密码	bool	首次登录设置密码 (激活账户)
resetPassword(userId, oldPwd, newPwd)	userId: 学号 oldPwd: 旧密码 newPwd: 新密码	bool	修改密码

表 2-7 AuthService 接口规范

3 系统实现

3.1 系统开发环境

类别	技术/工具	版本
开发语言	C++	C++17
GUI 框架	Qt	6.2.0+
数据库	MySQL	8.0
构建系统	CMake	3.16+
IDE	Qt Creator / Visual Studio	最新版
版本控制	Git	2.x
操作系统	Windows 10/11	64 位

表 3-1 开发环境技术栈

3.2 程序编码

本系统采用前后端分离的开发模式。本人主要负责前端 UI 部分的开发，包括客户端和管理端的界面设计、样式美化、页面跳转逻辑以及与后端接口的对接。

3.2.1 后端工作概述

- DAO 层：实现了 UserDao、GearDao、StationDao、RecordDao 四个数据访问类，封装了对 MySQL 数据库的 CRUD 操作
- 连接池：使用 ConnectionPool 单例类管理数据库连接，采用线程本地存储确保线程安全
- Service 层：实现了业务逻辑，包括认证、借还、费用计算等核心功能

3.2.2 前端工作详述

我主要负责前端 UI 部分的开发，包括客户端和管理端的界面设计与实现。

1. 全局样式系统设计

为了保证界面风格统一，我设计了一套完整的 QSS 样式系统，采用深蓝渐变主题：

```
01 // Styles.h - 配色方案定义
02 namespace Colors {
03     constexpr const char* Primary = "#667eea";           // 主色调-深蓝
04     constexpr const char* PrimaryDark = "#764ba2";       // 深色渐变
05     constexpr const char* Success = "#00d68f";           // 成功-绿色
06     constexpr const char* Warning = "#ffaa00";           // 警告-黄色
07     constexpr const char* Danger = "#ff3d71";            // 危险-红色
08 }
09
10 // 全局样式 - 深蓝渐变背景
11 inline QString globalStyle() {
12     return QStringLiteral(R"(
13         QMainWindow {
```

```

14         background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
15             stop:0 #667eea, stop:1 #764ba2);
16     }
17     // ... 更多样式定义
18     )");
19 }

```

2. 客户端主窗口实现

客户端采用 QStackedWidget 实现多页面切换，MainWindow 作为页面调度器：

```

01 // MainWindow.cpp - 页面初始化
02 void MainWindow::setupUi() {
03     m_stack = new QStackedWidget(this);
04
05     // 创建所有页面实例，注入所需的 Service
06     m_welcomePage = new WelcomePage(this);
07     m_userInputPage = new UserInputPage(m_authService.get(), this);
08     m_dashboardPage = new DashboardPage(m_stationService.get(), this);
09     m_borrowPage = new BorrowPage(m_borrowService.get(),
10 m_stationService.get(), this);
11
12     // 按顺序添加到 StackedWidget
13     m_stack->addWidget(m_welcomePage);        // 索引 0
14     m_stack->addWidget(m_cardReadPage);        // 索引 1
15     // ... 共 11 个页面
16 }
17
18 // 信号槽连接 - 实现页面间通信
19 void MainWindow::setupConnections() {
20     // 欢迎页 -> 刷卡页
21     connect(m_welcomePage, &WelcomePage::startClicked, this, [this]()
22 {
23         switchPage(Page::CardRead);
24     });
25
26     // 登录成功 -> Dashboard
27     connect(m_loginPage, &LoginPage::loginSuccess, this,
28 &MainWindow::onLoginSuccess);
29
30     // 借还操作完成 -> 刷新用户余额
31     connect(m_borrowPage, &BorrowPage::operationCompleted, this,
[ this]() {
    refreshUserData();
});
}

```

3. 槽位组件 SlotItem 设计

SlotItem 是自定义的 Qt 组件，用于展示借还页面的 12 个槽位：

```
01 // SlotItem.cpp - 状态刷新
02 void SlotItem::refreshStyle() {
03     QString indicatorColor;
04     QString borderColor;
05
06     switch (m_state) {
07     case State::Available:
08         indicatorColor = "#2ecc71"; // 绿色 - 可借
09         borderColor = "#2ecc71";
10         break;
11     case State::Empty:
12         indicatorColor = "#bdc3c7"; // 灰色 - 空槽可还
13         borderColor = "#bdc3c7";
14         break;
15     case State::Maintenance:
16         indicatorColor = "#e74c3c"; // 红色 - 故障
17         borderColor = "#e74c3c";
18         break;
19     }
20
21     // 设置状态指示器样式
22     m_statusIndicator->setStyleSheet(QStringLiteral(
23         "background-color: %1; border-radius:
24 2px;").arg(indicatorColor));
25
26     // 强制刷新组件样式
27     setStyleSheet(QStringLiteral(
28         "SlotItem { background-color: white; border: 2px solid %1; }")
29         .arg(borderColor));
30 }
```

4. 管理端侧边栏导航实现

管理端采用侧边栏+内容区的布局，通过静态辅助函数创建统一风格的侧边栏：

```
01 // 创建侧边栏辅助函数
02 static QWidget *createSidebar(QWidget *parent, AdminMainWindow *window,
03 int activePage) {
04     auto *sidebar = new QWidget(parent);
05     sidebar->setMinimumWidth(200);
06     sidebar->setStyleSheet(Styles::adminSidebar()); // 深蓝渐变背景
07
08     // 创建导航按钮
```

```

09     auto *btnDashboard = new QPushButton(QObject::tr(" 首页概览"),
10 sidebar);
11     auto *btnGear = new QPushButton(QObject::tr(" 雨具管理"), sidebar);
12     auto *btnUser = new QPushButton(QObject::tr(" 用户管理"), sidebar);
13     auto *btnOrder = new QPushButton(QObject::tr(" 订单流水"), sidebar);
14
15     // 根据 activePage 参数设置当前页面按钮的激活样式
16     btnDashboard->setStyleSheet(activePage == 1 ?
17         Styles::Buttons::sideNavActive() :
18 Styles::Buttons::sideNav());
19
20     return sidebar;
21 }

```

5. 自动刷新机制

借还页面和管理端均实现了定时自动刷新:

```

01 // BorrowPage.cpp - 自动刷新
02 BorrowPage::BorrowPage(...) {
03     m_refreshTimer = new QTimer(this);
04     connect(m_refreshTimer, &QTimer::timeout, this,
05 &BorrowPage::refreshSlots);
06 }
07
08 void BorrowPage::startAutoRefresh() {
09     if (m_refreshTimer && !m_refreshTimer->isActive()) {
10         m_refreshTimer->start(3000); // 每 3 秒刷新一次
11     }
12 }
13
14 // AdminMainWindow.cpp - 管理端刷新
15 AdminMainWindow::AdminMainWindow(...) {
16     m_refreshTimer = new QTimer(this);
17     connect(m_refreshTimer, &QTimer::timeout, this,
18 &AdminMainWindow::onRefreshTimer);
19     // 登录后启动, 每 5 秒刷新当前页面
20     m_refreshTimer->start(5000);
21 }

```

4 系统测试

4.1 用户借伞流程测试

业务流程: 欢迎页 → 刷卡/输入学号 → 登录验证 → 仪表板(选择站点) → 借伞页面(选择槽位) → 借伞成功

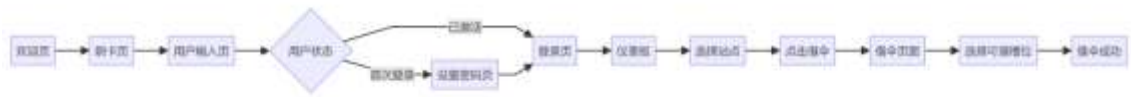


图 4-1 用户借伞流程图

下面是用户借伞流程测试结果，测试成功：



图 4-2 用户借伞流程测试结果图

4.2 用户还伞流程测试

业务流程：仪表板 → 点击还伞 → 还伞页面 → 选择空槽位 → 计算费用 → 扣款 → 还伞成功



图 4-3 用户还伞流程图

下面是用户还伞流程测试结果，测试成功：



图 4-3 用户还伞流程测试结果图

4.3 管理员操作流程测试

业务流程：管理员登录 → 首页概览(查看统计) → 雨具管理(修改状态) → 用户管理(重置密码) → 订单流水(查看记录)

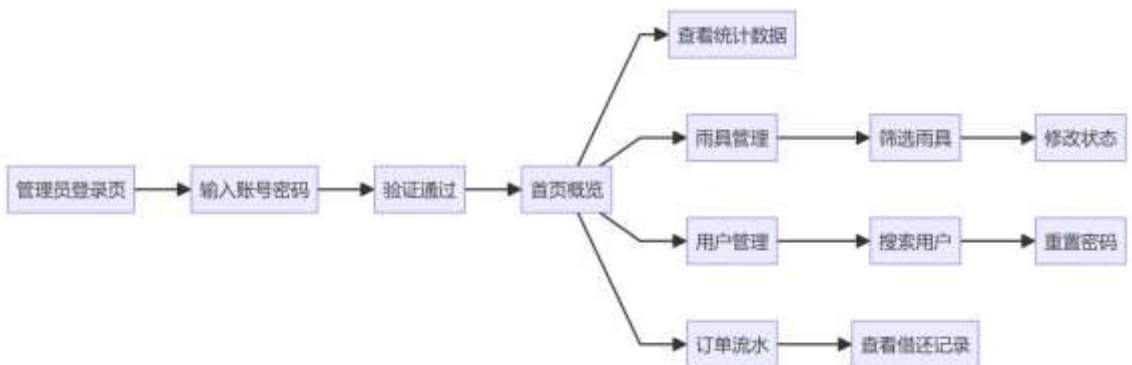


图 4-5 管理员操作流程图

下面是管理员操作流程测试结果，测试成功：

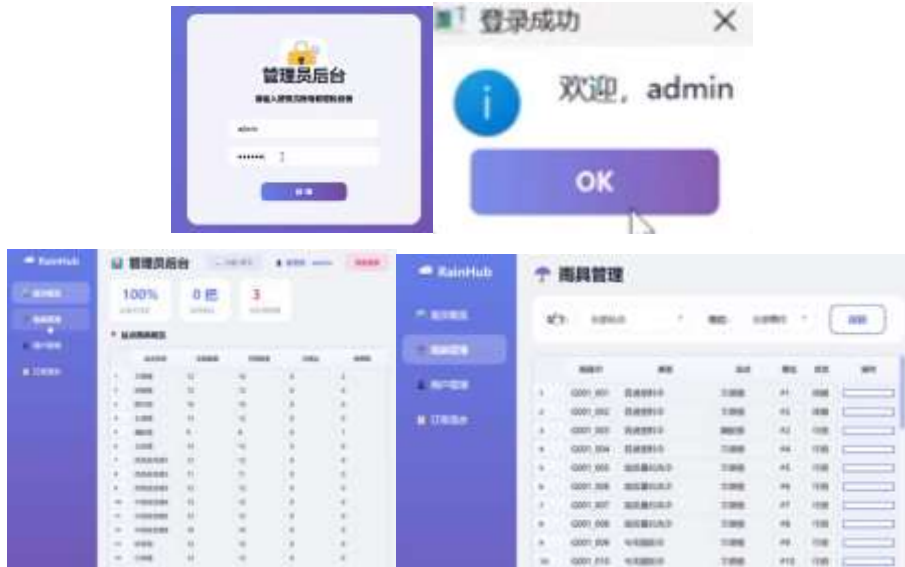


图 4-6 管理员操作流程测试结果图

5 总结与体会

5.1 遇到的问题及解决方法

问题描述	解决方案
Qt 样式表更新后不立即生效	使用 <code>style()->unpolish()</code> 和 <code>style()->polish()</code> 强制刷新，配合 <code>update()</code> 和 <code>repaint()</code>
页面切换时数据不同步	在 <code>switchPage()</code> 中根据目标页面调用对应的刷新函数
信号槽连接时 Lambda 捕获悬空引用	使用值捕获 <code>[gear = gear]</code> 代替引用捕获
QStackedWidget 页面索引与枚举不对应	严格保证 <code>addWidget()</code> 顺序与 <code>enum class Page</code> 定义一致
多线程数据库访问冲突	使用 <code>ConnectionPool::getThreadLocalConnection()</code> 获取线程

表 5-1 开发问题与解决方案

5.2 系统优点

- 完整的前后端架构
 - 美观的可视化界面（深蓝渐变主题、玻璃卡片效果）
 - 严格的 MVC 分层架构
- 现代 C++特性应用
 - 使用 `std::unique_ptr` 和 `std::shared_ptr` 智能指针管理内存
 - 使用 `std::optional` 处理可能为空的返回值
 - 使用 `enum class` 强类型枚举提高类型安全
- 工程化实践

- Git 版本控制
- CMake 跨平台构建系统
- Qt 信号槽机制实现松耦合使用

5.3 系统不足

1. 没来得及使用 Flask 开发 Web 版管理端，幸好目前的框架非常实用
2. 安全性设计不足，例如登录端本来想要用加密算法，没来得及学习
3. 我们目前的错误处理机制还是较简单的，异常情况处理不够完善
4. 没有能够实现完整的日志系统

5.4 收获与体会

问题解决能力提升：

- 学会了系统化的调试技巧，例如使用 Qt 自带的 `qDebug()` 输出以及断点调试
- 掌握了问题定位和分析方法，从现象追溯到根本原因，包括使用插件 SolarLint 还有使用数据库驱动 ddl 插件
- 积累了解决方案设计经验，权衡多种方案选择最优解

理论与实践结合：

- 将课堂学习的面向对象设计原则应用到实际项目中，对封装继承多态抽象这四个特性有了更深入的解读
- 深入理解了 MVC 架构模式的优势和实现方式，拓展了对架构的深入探索
- 体会到技术选型需要考虑项目规模、团队能力等多方面因素

工程思维培养：

- 认识到编程不只是实现功能，还要考虑可维护性、可扩展性
- 体会到代码质量的重要性，良好的命名和注释能大大降低维护成本
- 理解了文档和注释的价值，尤其是接口规范，这些对团队协作至关重要。由于没有提前准备统一的接口文档，在开发的过程中，识别接口就成了一个重体力活

5.5 致谢

感谢队友 xxx 在后端数据库 dao 层设计和 Service 层开发中的辛苦付出，当时确定主题就花费了很多心思，甚至有过一次推翻重来。我们在双方遇到问题的时候都进行了很多配合和帮助。

感谢老师在项目开发过程中的悉心指导和问题解答，让我们能够顺利完成这个项目。也感谢老师在我重修过程中对一些课程冲突灵活性调整的支持。

参考文献：

- [1] Qt Group. Qt 6.2 Documentation[EB/OL]. <https://doc.qt.io/qt-6/>, 2024.
- [2] Oracle Corporation. MySQL 8.0 Reference Manual[EB/OL]. <https://dev.mysql.com/doc/refman/8.0/en/>, 2024.
- [3] ISO/IEC. ISO/IEC 14882:2017 Programming Languages — C++[S]. Geneva: ISO, 2017.

小组编号	第 41 组	组长	xxx
学号	姓名	具体任务（分工）	
xxx	xxx	系统后端	
xxx	xxx	系统前端	